

config_endpoint

REFERENCE



API consumer



add, update, or delete an MQTT endpoint configuration on the reader

Overview

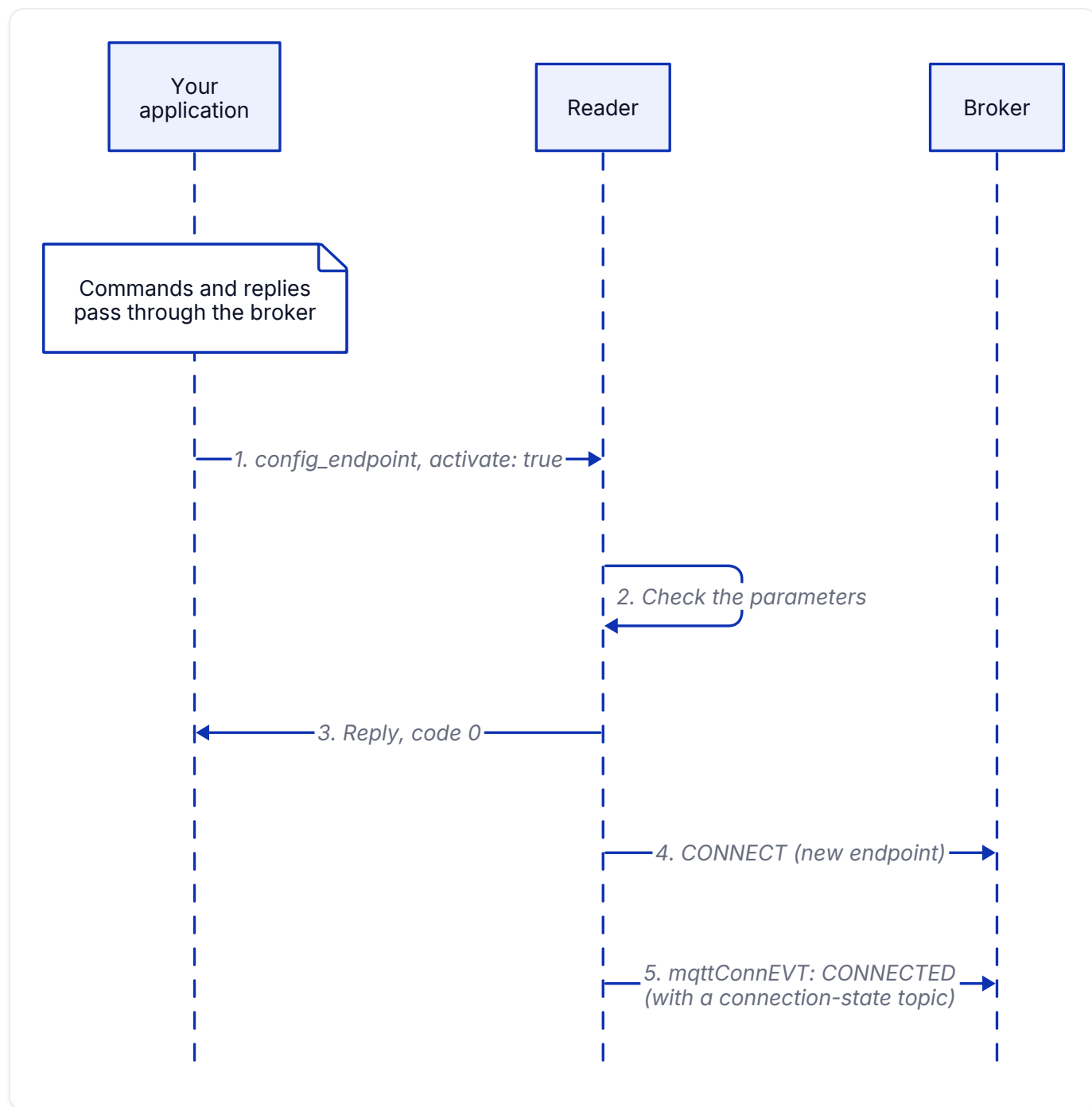
`config_endpoint` provisions the reader's MQTT endpoints — it **adds**, **updates**, or **deletes** an endpoint configuration (chosen by `epConfig.operation`). One payload defines an endpoint's type, protocol, broker `url` / `port`, TLS `verificationType` and certificates, MQTT `keepAlive` /session/reconnect settings, publish/subscribe topics, optional broker credentials, and per-endpoint event routing. It is a state-changing command on the device-management (MGMT) endpoint; the response is **status-only**. Read the result back with `get_endpoint_config`.

`config_endpoint` is **state-changing and can carry broker credentials** — and reconfiguring the endpoint that carries your own control session can sever it. See the session-sever callout in [Notes and Caveats](#) and the credential/TLS handling in [Request Semantics](#). A saved endpoint is reversible with `operation: delete`.

⚠ SAVING AN ENDPOINT DOES NOT CONNECT IT

An endpoint connects only when it is **active**. `activate: false` saves the endpoint; `activate: true` saves it and marks it active. With `activate: true`, the reader checks the endpoint's parameters. If they are valid, it disconnects from the current configuration and connects to the new endpoint; no reboot is needed. Wait for a `CONNECTED` `mqttConnEVT` on the endpoint's connection-state topic to confirm the connection. `get_endpoint_config` lists the endpoint under `activeEndpoints` as soon as it is stored as active, which does not show that it connected.

What happens when you add an endpoint with `activate: true`:



Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (writes/removes a saved endpoint; may activate it)
Reversibility	REVERSIBLE (remove with <code>operation: delete</code> ; change with <code>operation: update</code>)
Supported Operations	<code>add</code> (new), <code>update</code> (existing), <code>delete</code> (existing)

Property	Value
Required Request Fields	<code>command</code> , <code>requestId</code> , <code>epConfig</code> (with <code>operation</code> and <code>configuration</code>)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	v1.1 (API Version Reporting)
Related	get endpoint config , config events , install certificate , reboot

When to Use

Use `config_endpoint` to provision or maintain the reader's MQTT wiring after the initial `MDM` endpoint exists (that first endpoint is set up with 123RFID Desktop; all others are added remotely with `config_endpoint`).

- `add` — create a new endpoint (`MGMT`, `CTRL`, `DATA1`, ...). The `endpointName` must not already exist (duplicate → [code 10](#)).
- `update` — modify an existing endpoint (e.g. change `url`, or flip `activate`). The `endpointName` must already exist.
- `delete` — remove an existing endpoint (the contract's example sends only `endpointName` + `epType`).
- Pre-provision safely** — send `activate: false` to save an endpoint without activating it, then activate later with `update` . A saved endpoint does not connect until it is active.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

The `topic` values you configure inside `mqttParams` are **base** topics (e.g. `MGMT/clients/resp`). The reader constructs the full topic at runtime as `<tenantId>/<topic>/<serial>` — never include the tenant or serial in `topic` .

Credentials are presented once at CONNECT (and, for a configured endpoint, via `mqttParams`). The `requestId` you send is echoed in the response so you can pair reply to request.

See [Topics and Channels](#).

Key Fields

- `epConfig.operation` (request) — `add` / `update` / `delete`; determines the action.
- `configuration.endpointName` (request) — the endpoint's identity (create target / update / delete key).
- `configuration.epType` (request) — the endpoint's role (`MGMT`, `CTRL`, `DATA1`, ...).
- `configuration.activate` (request) — `true` saves the endpoint and marks it active, and the reader connects for it without a reboot if the parameters are valid; `false` only saves it.
- `configuration.securityParams` (request) — for `MQTT_TLS`; the referenced certificates must be pre-installed via [install certificate](#).
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The envelope is `command`, `requestId`, and the `epConfig` payload (`operation` + `configuration`). Pick an **operation** / **endpoint** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

Operation / endpoint

Add MGMT endpoint for MQTT TLS ▾

Request payload

Example: add, MGMT, MQTT_TLS

```
1      {
2        "command" : "config_endpoint" ,
3        "requestId" : "1233" ,
4        "epConfig" : {
5          "operation" : "add" ,
6          "configuration" : {
7            "endpointName" : "mgmt_tls" ,
8            "epType" : "MGMT" ,
9            "protocol" : "MQTT_TLS" ,
10           "activate" : true ,
11           "url" : "broker.ip.com" ,
12           "port" : 8883 ,
13           "verificationType" : "VERIFY_HOST_AND_PEER" ,
14           "qosCommon" : 1 ,
15           "tenantId" : "ZEBRA" ,
16           "mqttParams" : {
17             "keepAlive" : 300 ,
18             "cleanSession" : true ,
19             "reconnectDelayMin" : 5 ,
20             "reconnectDelayMax" : 60 ,
21             "publishTopics" : [
22               {
23                 "topic" : "MGMT/clients/resp" ,
24                 "qos" : 1 ,
25                 "retain" : false
26               } ,
27               {
28                 "topic" : "MGMT/clients/event" ,
29                 "qos" : 1 ,
30                 "retain" : false
31               } ,
32               {
33                 "topic" : "MGMT/clients/rfid" ,
34                 "qos" : 0 ,
35                 "retain" : true
36               }
37             ] ,
38             "subscribeTopics" : [
39               {
40                 "topic" : "MGMT/clients/cmnd" ,
41                 "qos" : 0
42               }
43             ]
44           } ,
45           "securityParams" : {
46             "format" : "PEM" ,
47             "caCertificateFile" : "mqtt_ca_cert" ,
48             "clientCert" : "mqtt_client_cert" ,
49             "clientKey" : "mqtt_client_key"
50           }
51         }
52       }
53     }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	config_endpoint	Fixed comma
2	requestId *	string	—	abc123	A unique iden echoed back
3	epConfig *	object	—	—	The endpoint payload. Req which also re and configu
4	operation *	enum	add delete update	add	add a new en existing one, Required with Request Sem
5	configura tion *	object	—	—	The endpoint Required with contract requi epType, prot url, verific port for ever its own delet examples ser Semantics).
6	endpoi ntName *	string	—	mgmt	The endpoint add/update/d
7	epType *	enum	MGMT MGMT_EVT CTRL DATA1 DATA2 SOTI MDM	MGMT	The endpoint is currently n
8	protoc ol *	enum	MQTT MQTT_TLS	MQTT	MQTT (plain, t MQTT_TLS (TI 8883). Both li unchanged vi get_endpoi Request Sem
9	activa te *	boolean	—	true	true saves th marks it activ its parameter valid, connect reboot. false (activate it lat
10	url *	string	—	soti.example.com	Broker addres

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
11	verification * Type	enum	VERIFY_NONE VERIFY_HOST_AND_PEER	VERIFY_HOST_AND_PEER	TLS verification. Use VERIFY_NONE in production, and VERIFY_HOST_AND_PEER in development.
12	port *	integer	—	1883	Broker port (1883 for MQTT, 883 for MQTTS).
13	qosCommon	integer	—	1	Default QoS level. 0 or 1, the level supported by the broker.
14	tenantId	string	—	ZEBRA	Tenant identifier of every on-warehouse contract. Send it without the contract's tenantId. Bounded at 4 characters, no overlong values.
15	mqttParams	object	—	—	MQTT connection parameters, including topics, and (optional) credentials.
16	keepAlive	integer	max 65535	300	Keep-alive interval in seconds.
17	cleanSession	boolean	—	true	Whether to start a new session.
18	reconnectDelayMin	integer	—	5	Minimum reconnect delay in seconds.
19	reconnectDelayMax	integer	—	60	Maximum reconnect delay in seconds.
20	clientId	string	—	zebra-rfd40-01	MQTT client identifier for connection.
21	username	string	—	mqttuser	Broker username. Requires username if the broker requires it. Redact it in logs.
22	password	string	—	—	Broker password. Requires password if the broker requires it. — a secret; not logged.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
23	publishTopics	array	—	—	Base topics to — at most code 25). Entire schema example responses, [[: connection example Will]. On DATA guidance placed at [1] — see Each item retained.
24	topic*	string	—	zebra/rfd40/01/data	Base topic on wire to <tenantId>, Do not include
25	qos*	integer	—	1	QoS for this packet the levels the
26	retain*	boolean	—	false	Whether the packet is a retained message.
27	subscribeTopics	array	—	—	Base topics that the client subscribes to returns code , requires topic
28	topic*	string	—	zebra/rfd40/01/data	Base topic on wire to <tenantId>, Do not include
29	qos*	integer	—	0	QoS for this packet the levels the
30	retain*	boolean	—	—	Retain flag for the message.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
31	securityParams	object	—	—	TLS parameters. Required with format. The certificates must be installed via install certificates .
32	format *	enum	PEM PFX	PEM	Certificate format. Supports PEM and PFX, currently supports PEM.
33	caCertificateFile	string	—	cloud-trusted-ca-certs.crt	Installed CA certificate file name (a name).
34	clientCert	string	—	client_cert.pem	Installed client certificate name.
35	clientKey	string	—	client_key.pem	Installed client key name.
36	eventConfiguration	object	—	—	Per-endpoint configuration. Same shape as eventConfiguration , selecting which endpoint emit heartbeat set. Are functional Semantics.
37	antenna	boolean	—	false	set to true to enable antenna events
38	terminalConnection	boolean	—	false	set to true to enable terminal connection/disconnection events
39	firmwareUpdate	boolean	—	true	set to true to enable firmware update events
40	gpi	boolean	—	false	set to true to enable GPI events
41	network	boolean	—	true	set to true to enable network events
42	exceptions	boolean	—	false	set to true to enable exceptions events
43	ntp	boolean	—	true	set to true to enable NTP alerts

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
44	userApp	boolean	—	false	set to true to enable events
45	heartbeat	boolean	—	false	set to true to enable heartbeat
46	power	boolean	—	true	set to true to enable power alerts
47	battery	boolean	—	true	set to true to enable battery alerts
48	temperature	boolean	—	false	set to true to enable temperature alerts
49	fileDownload	boolean	—	true	set to true to enable file download alerts
50	cpuUsage	boolean	—	false	set to true to enable CPU usage alerts
51	flashUsage	boolean	—	false	set to true to enable flash usage alerts
52	ramUsage	boolean	—	false	set to true to enable RAM usage alerts
53	heartbeatConfiguration	object	—	—	Heartbeat set heartbeat configuration
54	interval	integer	max 65535	100	Seconds between most 65,535.
55	inventoryStatus	boolean	—	true	set to true to enable inventory status as part of message

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
56	batteryStatus	boolean	—	true	set to true to report battery status as part of heartbeat
57	userAppStatus	boolean	—	—	set to true to report user app status as part of heartbeat message
58	cpuThreshold	integer	—	0	cpu usage value. If value is greater than alert will be triggered
59	ramThreshold	integer	—	0	ram usage value. If value is greater than alert will be triggered
60	flashThreshold	integer	—	0	flash usage value. If value is greater than alert will be triggered
61	temperatureThreshold	integer	—	0	temperature threshold. If value is greater than which alert will be triggered
62	batching	object	—	—	Report batching configuration. Optional; when enabled, the reader's built-in batching is used. A report is published when the batch limit is reached
63	reportingInterval	integer	1000–60000	5000	Reporting time interval, in milliseconds. The minimum is 1000ms
64	maxPayloadSizePerReport	integer	1024–5120	1024	Maximum payload size in bytes. 0 falls back to the default minimum, 1024 bytes

Request Semantics

Modes. `epConfig.operation` selects the action. The contract requires the same seven `configuration` fields for every operation — `endpointName`, `epType`, `protocol`, `activate`, `url`, `verificationType` and `port` — but its own `delete` example sends only `endpointName` and `epType`, and its `update` example omits `verificationType` and `port`. A validator built strictly from the contract rejects both examples; this is a known issue in the source schema:

- `add` — creates a new endpoint. Supply the full `configuration` (the 7 fields `add` requires — `endpointName`, `epType`, `protocol`, `activate`, `url`, `verificationType`, `port` — plus optional `mqttParams` / `securityParams` / `eventConfiguration`). Fails with [code 10](#) if the `endpointName` already exists.
- `update` — modifies an existing endpoint (matched by `endpointName`). The contract's example sends `endpointName` and `epType` plus the fields being changed.
- `delete` — removes an existing endpoint. The contract's example sends only `endpointName` and `epType`.

`add` is **not idempotent**. If a response is lost (e.g. a timeout), the endpoint may already have been created — **read back with `get_endpoint_config` before retrying** rather than blindly re-sending, or the retry will be a duplicate `add`.

- **Payloads can carry broker credentials.** `mqttParams.username` / `password` are supplied in the request when the broker requires them — masked in the example on this page. Redact them in logs, tickets, and screenshots, and send `config_endpoint` only over secured channels. (Certificate fields — `caCertificateFile` / `clientCert` / `clientKey` — are **reference names** of already-installed certificates, not key material.)
- **Optional MQTT connection fields.** `mqttParams.clientId` sets the MQTT client ID used for the broker connection; `cleanSession` defaults to `true`.
- **TLS needs pre-installed certificates.** For `protocol: MQTT_TLS`, the `securityParams` `caCertificateFile` / `clientCert` / `clientKey` must reference certificates already installed via [install_certificate](#). Only `format: PEM` is supported today. The protocol literal is spelled `MQTT_TLS` in both directions — one enum governs the request and the `get_endpoint_config` read-back — so send and match it exactly.
- **Topic slots are positional.** `publishTopics` accepts **at most 3** entries (exceeding → [code 25](#)) and `subscribeTopics` **at most 1** (exceeding → [code 26](#)); both arrays are optional — the contract's own `DATA1` read-back returns `subscribeTopics: []`. The three publish slots carry command **responses** (`.../clients/resp`), **events** — heartbeat, alerts (`.../clients/event`), then the endpoint's **connection-event** topic (`.../clients/rfid`, which carries `mqttConnEVT`); `subscribeTopics[0]` is **command ingress**, where the host publishes and the reader subscribes (`.../clients/cmdnd`). Position is what routes — the reader binds each entry by **index, not by base name** — so keep the configured order, leave omitted indices unused, and match a `get_endpoint_config` read-back by index rather than by topic name. Configure only the base `topic`; the reader wraps it as `<tenantId>/<topic>/<serial>`.
- **DATA endpoints: put the tag-data topic at `publishTopics[1]`.** On `epType DATA1` the reader streams tag data from the **second** publish entry (`DATA2` is currently not supported), so configure an unused placeholder at `[0]` and the tag-data topic (e.g. `DATA/clients/dataevent`) at `[1]`: `[{ "topic": "DATA/clients/unused", "qos": 0, "retain": false }, { "topic": "DATA/clients/dataevent", "qos": 1, "retain": false }]`. Every `DATA1` example on this page uses this two-entry layout. A single-entry array is the trap: the one entry lands in the **reply** slot, the `add` still succeeds, and no tag data ever arrives. A read-

back showing `activate: true` shows only that the endpoint is stored as active; it does not show that the endpoint connected or that tag data is flowing.

- `verificationType`: **use** `VERIFY_HOST_AND_PEER`. It verifies the broker's certificate chain and that the certificate matches the host. `VERIFY_NONE` turns verification off; use it only on an isolated test network, and never on an `MQTT_TLS` endpoint, which rejects it with [code 37](#). `get_endpoint_config` reads back the value you sent, so a client doing read-modify-write can echo it.
- **Event routing.** `configuration.eventConfiguration` uses the same shape as `config_events`; only 7 of its toggles are functional today (the rest are reserved). See the Event routing fields table on the [get_endpoint_config](#) page for the full supported-vs-reserved split.

Topic slots by `epType`. How the positional layout above lands per endpoint type. The contract's examples show the command/response layout; the data-endpoint layout (†) is firmware guidance that the contract does not state:

<code>epType</code>	<code>publishTopics[0]</code>	<code>publishTopics[1]</code>	<code>publishTopics[2]</code>	<code>subscribeTopics[0]</code>
<code>MGMT</code> · <code>MDM</code> · <code>SOTI</code>	command responses (<code>.../resp</code>)	events (<code>.../event</code>)	connection events (<code>.../rfid</code>)	command ingress (<code>.../cmnd</code>)
<code>CTRL</code>	command responses (<code>.../resp</code>)	stored, not used for events	connection events (<code>.../rfid</code>)	command ingress (<code>.../cmnd</code>)
<code>MGMT_EVT</code>	not used	not used	Last Will	events are currently published here (a firmware defect)
<code>DATA1</code>	unused placeholder †	tag-data stream (<code>.../dataevent</code>) †	—	— (does not subscribe)

The command/response endpoints (`MGMT` , `MDM` , `SOTI` , `CTRL`) share the `[0]` and `[2]` layout; on `CTRL` , slot `[1]` is stored but not used for events. A `MGMT_EVT` endpoint currently publishes its events on its subscribe topic instead of `publishTopics[1]` , which the API owners record as a firmware defect; see [Endpoints and Topic Slots](#). Match read-back entries by index, not by base name — the `resp` / `event` / `rfid` names are conventions.

† Tag data binds to `publishTopics[1]` , so a data endpoint needs a throwaway entry at `[0]` to push the real topic into slot `[1]` — see the DATA-endpoint bullet above.

Response Payload

`config_endpoint` returns a **status-only** envelope — `command` , `requestId` , `apiVersion` , and `response` . There is no configuration echo; confirm the change with `get_endpoint_config` .

Response payload

```

1  {
2    "command": "config_endpoint",
3    "requestId": "abc123",
4    "apiVersion": "V1.1",
5    "response": {
6      "code": 0,
7      "description": "Success"
8    }
9  }

```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	config_endpoint	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Status-only result — config_endpoint echoes no configuration.
5	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
6	description *	string	—	Success	Human-readable status.

Response Codes

config_endpoint returns response.code. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the endpoint was added, updated, or deleted.

Code	Description
10	Configuration already exists (<code>IOT_ERROR_CONFIG_ALREADY_EXIST</code>) — an <code>add</code> used an <code>endpointName</code> that already exists. Use <code>update</code> , or delete it first.
23	Invalid enum value (<code>IOT_ERROR_INVALID_ENUM</code>) — a field (e.g. <code>operation</code> , <code>epType</code> , <code>protocol</code> , <code>verificationType</code> , <code>format</code>) is outside its allowed set.
25	Max 3 publish topics exceeded (<code>IOT_ERROR_PUBLISH_TOPICS_EXCEEDED</code>) — more than 3 <code>publishTopics</code> .
26	Max 1 subscribe topic exceeded (<code>IOT_ERROR_SUBSCRIBE_TOPIC_EXCEEDED</code>) — more than 1 <code>subscribeTopics</code> .
27	Invalid tenant ID length (<code>IOT_ERROR_INVALID_TENANTID_LENGTH</code>) — <code>tenantId</code> exceeds the allowed length.
33	Endpoint already configured (<code>IOT_ERROR_ENDPOINT_ALREADY_CONFIGURED</code>) — an endpoint with the same configuration already exists.
35	Can't modify an active endpoint (<code>IOT_ERROR_CANT_MODIFY_ACTIVE_ENDPOINT</code>) — the target endpoint is currently active. Deactivate it before modifying it.
37	Invalid verification type for MQTT TLS (<code>IOT_INVALID_MQTT_TLS_VERIFICATION_TYPE</code>) — <code>protocol: MQTT_TLS</code> was sent with <code>verificationType: VERIFY_NONE</code> . Use <code>VERIFY_HOST_AND_PEER</code> .
101	Default error — generic catch-all. See the duplicate- <code>add</code> note below.

Duplicate `add` may return the generic 101. The documented code for adding an `endpointName` that already exists is 10 ("Configuration already exists"), but the firmware has been observed returning the generic 101 ("Default error") for a duplicate `add` instead of 10. Treat a 101 after a re-sent `add` as a probable duplicate and verify with `get_endpoint_config` .

Examples

Publish a `config_endpoint` request; the device replies with the status-only envelope under [Response Payload](#). Every contract example — each `add` , the `delete` and the partial `update` — is in the **Operation / endpoint** dropdown under [Request Payload](#). The [examples](#) below show what the dropdown does not.

A **plain-MQTT** `DATA1` `add` with broker credentials (`activate: true`), the tag-data topic at publish slot [1] with an unused placeholder at [0] — the layout the firmware guidance under [Request Semantics](#) describes — and no TLS:

Request — add (plain MQTT, with credentials)

```
1  {
2    "command": "config_endpoint",
3    "requestId": "abc123",
4    "epConfig": {
5      "operation": "add",
6      "configuration": {
7        "endpointName": "dataEP",
8        "epType": "DATA1",
9        "protocol": "MQTT",
10       "activate": true,
11       "url": "broker.example.com",
12       "verificationType": "VERIFY_NONE",
13       "port": 1883,
14       "qosCommon": 1,
15       "tenantId": "zebra",
16       "mqttParams": {
17         "keepAlive": 300,
18         "cleanSession": true,
19         "reconnectDelayMin": 50,
20         "reconnectDelayMax": 500,
21         "username": "*****",
22         "password": "XXXXXXXX",
23         "publishTopics": [
24           {
25             "topic": "DATA/clients/unused",
26             "qos": 0,
27             "retain": false
28           },
29           {
30             "topic": "DATA/clients/dataevent",
31             "qos": 1,
32             "retain": false
33           }
34         ]
35       }
36     }
37   }
38 }
```

A duplicate `add` returns `code 10` (or, on some firmware, the generic `101` — see the note above):

Response — duplicate add (code 10)

```
1  {
2    "command": "config_endpoint",
3    "requestId": "abc123",
4    "apiVersion": "V1.1",
5    "response": {
6      "code": 10,
7      "description": "Configuration already exists"
8    }
9  }
```

Notes and Caveats

⚠ RECONFIGURING THE ACTIVE CONTROL ENDPOINT CAN SEVER YOUR SESSION

Activating a *new* endpoint of a different role coexists with the endpoint carrying your control session (e.g. activating a `DATA1` endpoint does not drop the `MDM` / `MGMT` control channel). But changing the broker `url` / `port` / `protocol` of — or deleting — the endpoint that carries your **own** control session can sever it. When in doubt, save with `activate: false` and switch deliberately.

Because the response is status-only, **verify the result with** `get_endpoint_config` — confirm the endpoint appears in `savedEndpoints.epNames[]` and (if `activate: true`) under `activeEndpoints`. A `CONNECTED mqttConnEVT` on the endpoint's connection-state topic confirms that it connected. For the concept-level view, see [Endpoints and Topic Slots](#).

Related:  [Endpoints and Topic Slots](#) ·  `get_endpoint_config` ·  `config_events` ·  `install_certificate` ·  [Response Codes](#)